CHAPTER

18

Learning electron densities

*Bruno Cuevas-Zuviría**

Centro de Biotecnología y Genómica de Plantas (CBGP, UPM-INIA), Universidad Politécnica de Madrid (UPM), Instituto Nacional de Investigación y Tecnología Agraria y Alimentaria (INIA), Madrid, Spain; NASA Center for Early Life and Evolution, University of Wisconsin-Madison, Madison, WI, United States; Department of Bacteriology, University of Wisconsin-Madison, Madison, WI, United States

***Corresponding author: E-mail address: bruno.czuviria@upm.es**

Abstract

Electron positions are intrinsically probabilistic. The electron density function ($\rho(r)$) condensates the information of the wave function into a three-dimensional representation of the probabilities of finding a charge in a region of space. Its machine learning prediction opens the gates of physically motivated quantum mechanics–machine learning, given that it grants access to many different aspects of molecular systems. In this chapter, we will explore some of the fundamental theoretical concepts of electron density, the foundations of the methods developed in the literature, some insights about how to calculate and integrate the electron density, a study case around the training of an A3MDnet model for the prediction of sugar's electron density, and the inference of electron density using DeepDFT.

Keywords: Electron density, Machine learning, Computational chemistry, Tensorial properties, Physical motivation

Introduction

A property to rule them all

As explained in the first chapters, electrons behave according to the laws of quantum mechanics (QM). The most important consequence is the probabilistic relationship between the system states and system observations.

Finding the electron charge in a region of space is an observation of a quantum system, so we can only recover a probability for this event. This probability is given in QM by the square

Copyright © 2023 Elsevier Inc. All rights reserved.

(more properly, the product by its complex conjugate) of the amplitude of the wave function. This operation defines the *electron density* $\rho(\mathbf{r})$ at a point in space $\mathbf{r}$:

$$\rho(\mathbf{r}) = \psi^*(\mathbf{r})\psi(\mathbf{r}) \tag{1}$$

In systems containing more than one electron, we need to previously integrate the space positions of all the other electrons:

$$\rho(\mathbf{r}) = \int \psi^*(\mathbf{r}, \mathbf{r}_2, \mathbf{r}_3 \ldots)\psi(\mathbf{r}, \mathbf{r}_2, \mathbf{r}_3, \ldots)\, d\mathbf{r}_2\, d\mathbf{r}_3 \ldots \tag{2}$$

Because electrons are indistinguishable fermions, this integral can be expressed as just a sum of squared molecular orbitals $\phi_i(\mathbf{r})$ (real functions):

$$\rho(\mathbf{r}) = \sum_i^{MO} n_i \, |\phi_i(\mathbf{r})|^2 \tag{3}$$

where n_i is the orbital occupancy, that is, the number of electrons in that orbital and MO is the number of molecular orbitals. A simple derivation of this equation can be obtained by integrating a dummy wave function $\psi(r_1, r_2) = \frac{1}{\sqrt{2}}(\phi_1(r_1)\phi_2(r_2) - \phi_2(r_1)\phi_1(r_2))$ with Eq. (2).

The electron density is a key function of molecular systems. Although the wave function contains all sources of information, it is defined as a 4N-dimensional [three-dimensional (3D) space plus spin for the N electrons in the system] complex-valued function which turns most calculations burdensome. On the contrary, the electron density is defined only in the 3D real space—the one we are most familiar with. The electron density plays a multifaceted role in quantum chemistry by condensing many properties of the electronic wave function in just the three coordinates of space.

- **In theory, we could find the energy of many-body electronic problems from their electron density**. According to Hohenberg–Kohn theorems [1,2] (Chapter 2 *Density functional theory*), the energy of an electronic system under an external potential is the lower bound of an electron density functional. Thus, by minimizing such a density functional, we could solve electronic structure problems.

  The exact functional remains unknown. However, the application of density functionals combined with wave function calculations has given place to the most widely employed type of QM calculations: the Kohn–Sham density functional theory (KS-DFT) [3,4].
- **The electron density allows the definition of important molecular properties**. The most straightforward case is the calculation of the moments of electron density. However, density can also be applied to address more abstract concepts, such as partitioning space into atoms or defining covalent bonds [5].
- **We can directly interact with electron density**. The electron density is an observable, so it can be directly measured. The best examples might be X-ray diffraction or cryogenic electron microscopy [6].

Paraphrasing J. R. R. Tolkien, we find that electron density is a property to rule them all.

Before diving into more specific aspects of machine learning (ML)-based prediction of electron density, it is important to understand that the goal of this chapter is the modeling of a function of 3D real space. Although we do not know its exact analytical shape for many-electron systems, we know some important aspects that the function must fulfill as well as others that $\rho(\mathbf{r})$ follows in isolated atoms.

First, according to the explanation above, $\rho(\mathbf{r})$ behaves as a probability distribution multiplied by the number of electrons N of the system. This means that it must be normalized: the 3D space integral of the electron density must then be equal to N:

$$\int \rho(\mathbf{r}) d\mathbf{r} = N \tag{4}$$

Probability distributions can only be positive: a negative probability would be meaningless. Therefore, its normalization must be fulfilled either by bounding the electron density on a given region or by displaying a vanishing decaying asymptotic behavior with distance ($\lim_{r\to\infty} \rho(r) = 0$). Some theorems prove that electron density must decay exponentially at long distances [7].

We might assume that because the electron density is a function with a physical meaning, it must be continuous and smooth. However, $\rho(\mathbf{r})$ gives us a little surprise here: although its decaying behavior is continuous and smooth, the value of the electron density at the nuclear positions in a molecular system shows *cusps*: discontinuities on the derivative. These discontinuities are associated with the singularity of the nuclear potential at nuclei (Kato's cusp theorem [8]).

Combining the fact that the electron density must show asymptotic decay with distance and a cusp at nuclei makes exponential functions the most likely ansatz for electron density representation. Besides, the exact electron density of mono-electronic systems such as hydrogen atoms provided by the Schrödinger equation is given by the exponential function

$$\rho_H(r) = \frac{1}{\pi} e^{-2r} \tag{5}$$

However, different *Ansätze* are usually employed for the sake of computational efficiency. The most important approaches are the use of Gaussian basis sets and the use of real space grids. Fig 1A shows the electron density of a small aromatic organic molecule for different isosurfaces.

In what follows, we will explain some of the recent approaches to electron density learning (Fig. 1B). Methods aiming directly at density prediction can be *soft-divided* into DFT-inspired algorithms, numerical approaches, and analytical approaches. The first ones employ ML inside DFT formalisms, and the electron density is predicted either directly or indirectly. Numerical methods employ a generalized ML ansatz (e.g., a deep neural network) to evaluate electron density. Given their numerical nature, property calculations require accurate numerical integration. Analytical density methods parameterize a basis set of functions. Therefore, they might suffer from basis-set limitations as standard QM methods, but they might also ease some calculations. We can consider electron density a task related with wave function prediction, given that both objects are interrelated by Eq. (3).

Topology

Before explaining how to learn electron density, we would like to make a small digression on a theory that highlights the multifaceted role of electron density: the *Atoms in Molecules* (AiM) theory, also called *Quantum Topology Theory* (QTT) [5,11]. This theory aims to define a wide number of relevant molecular properties through a rigorous partition of the electron density, which in turn leads to proving the existence of *atoms* inside molecules.

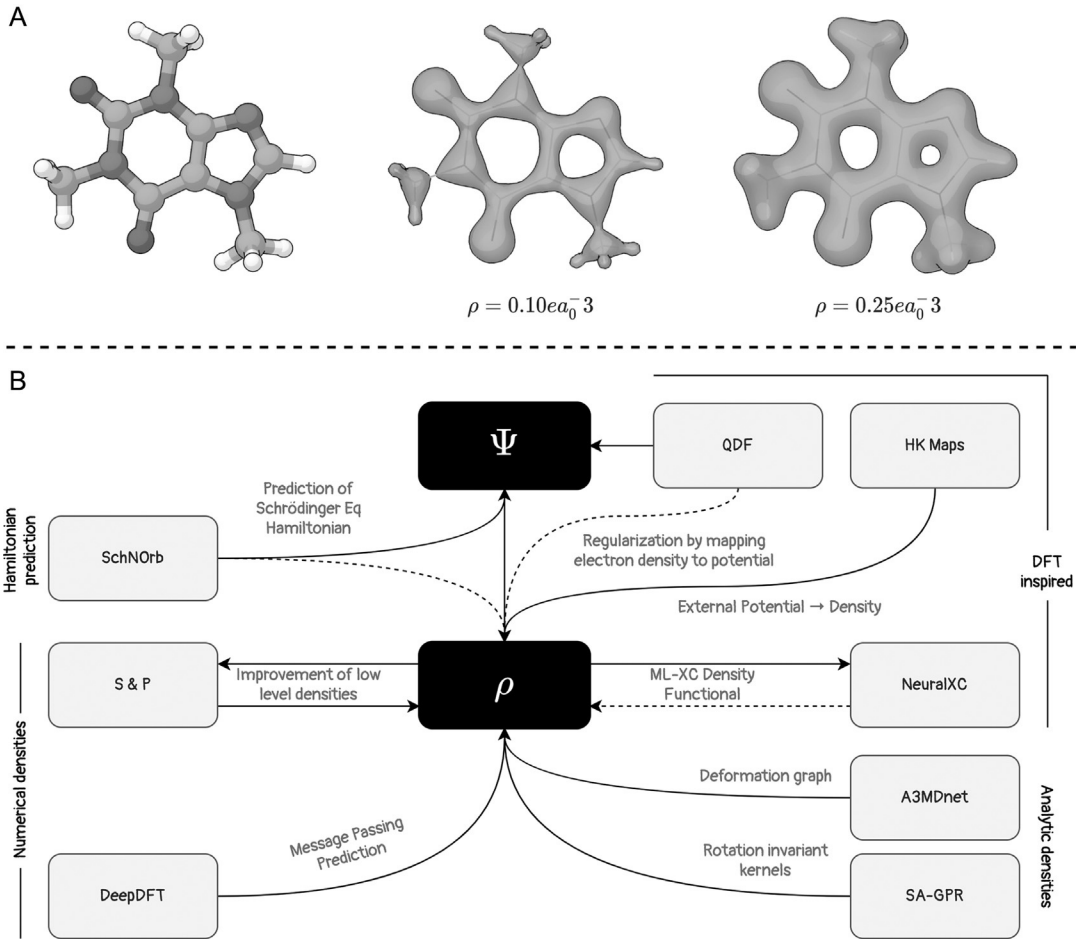


FIG. 1 (A) Electron density isosurfaces (right) of an organic molecule (left). Each isosurface consists of a set of points with the same electron density values. (B) Methods predicting electron density. We placed methods in groups according to their similarities, and we used different kinds of lines to remark their relationship with different prediction outcomes. Continuous lines indicate electron-density-aimed methods, whereas dashed lines indicate indirect learning of the electron density. For instance, Quantum Deep Field (QDF) [9] is not trained to reproduce $\rho(r)$, but it is able to perform qualitative electron density predictions. SchNOrb is trained to predict the wave function, but once we have a wave function, we can predict the electron density. S&P is the method from Pande and Sinitskiy [10]. All the other methods will be presented in the following paragraphs. *Credit: Bruno Cuevas.*

Proving the existence of atoms might sound strange. After all, the concept of atom was already accepted at the beginning of the 21st century. However, the conventional QM framework does not bring a specific definition for it: molecules do not contain atoms but a collection of nuclei and electrons subject to some attractive and repulsive interactions. AiM proves a rigorous partition of the space into atomic basins. For this, the theory employs the gradient paths of the $\rho(\mathbf{r})$ function to generate meaningful partitions of the electron density. A gradient path is a curve to which electron density gradients are tangential at each of its points (Fig. 2).

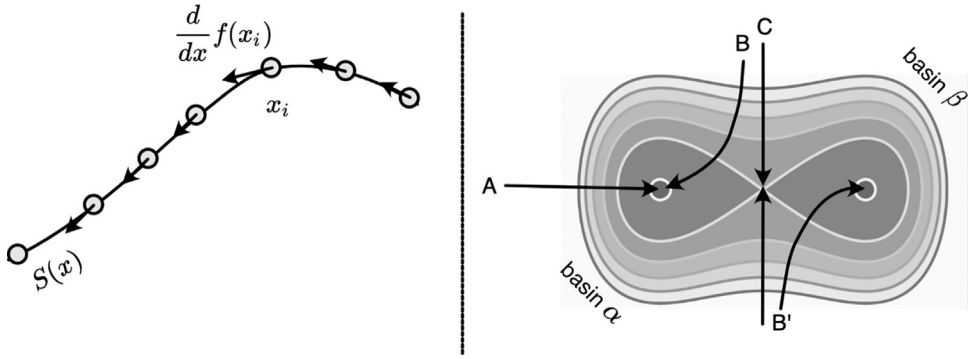


FIG. 2 (Left) Gradient path of a given function $f(x)$. For soft functions, the gradient path draws a unique trajectory that only depends on the starting point. AiM theory divides the 3D space according to where a gradient path starting at a given point leads. (Right) Dummy example of gradient paths on a double-well potential, which has some similarities with diatomic molecules. The path starting from A leads to a given point that we might call attractor A. Paths starting from B and B′ lead to different attractors but with a large degree of curvature. Paths starting at some points between the two attractors would not lead to any of the attractors. The set of points that converge to the same attractor makes an atomic basin, whereas the points that lie between make a surface that separates both basins. *Credit: Bruno Cuevas.*

If we move along the trajectories generated by the gradients of $\rho(\mathbf{r})$ (gradient paths), we will find that from most of the points, we will finish our motion in a nucleus of an atom. The collection of points whose gradient paths point toward a given nucleus is called the atomic basin of that nucleus in QTT so that we can consider that collection of points an *atom in a molecule*. This could be just another way of splitting 3D space employing the electron density, competing with Hirshfield or Minimal Basis Iterative Stockholder [12] methods used in QM. However, QTT goes further by showing that the virial theorem proves that splitting the kinetic energy of the electrons contained in each of these basins allows to retrieve the total kinetic energy. Thus, these basins have a physical meaning, and they can indeed be considered as *Atoms in Molecules*.

AiM theory does not only deal with atoms but employs the electron density derivatives to find different regions and points on space with singular properties. Most of them are related with ill-defined concepts in QM such as atoms or chemical bonds. Studying how these singular points and regions change upon perturbations is an illustrative way to understand changes on molecular structure. AiM theory is not the only theory employing the electron density as a source of information to define chemical concepts. The application of information theory to study chemical systems has also received attention lately [13].

Methods

This section will have two parts: an initial part, about the prediction of the electron density using different approaches, and a second part, with basic concepts about how to calculate the electron density from a QM wave function or how to integrate electron density in 3D space.

Prediction of the electron density

As mentioned above, electron density is a function of 3D space with an exponential-like decay. Therefore, any method aiming at its reproduction must generate a molecule-tailored map $\mathcal{F}(M)$ that joins 3D coordinates $\mathbf{r}$ with a real density value.

$$\mathbf{r} \overset{\mathcal{F}(M)}{\rightarrow} \rho \tag{6}$$

There might be infinite ways of building such maps. However, most methods in the literature can be split in either of the two approaches: a numerical one, which aims to predict electron density at positions with an ML ansatz, and an analytic approach, which aims to predict electron density at the space of functions. These two approaches can be compared against their DFT counterparts: numerical-grid-based methods (e.g., those implemented in SIESTA) and basis set methods (e.g., those used by Gaussian, PySCF, …).

Besides their differences on how they relate with $\rho(r)$, methods differ on how they deal with orientation. Electron density is equivariant to rotation. This means that ML methods aiming to predict electron density must encode rotational invariance somehow. Some methods only map the electron density locally from another function (e.g., the external potential, HK-maps [14]), so their values should not be affected by rotations. Others only employ distance relationships between atoms (e.g., DeepDFT, A3MDnet). SA-GPR [15,16] and XCnet [17] employ rotation-invariant representations based on spherical harmonics. Moreover, some methods do not implement rotational invariance, so they must learn it through data augmentation (e.g., SchNOrb [18,19]).

Numerical methods

Numerical methods describe the electron density employing a generalized ML ansatz to describe the electron density in a set of points of 3D space. Given that an ML ansatz allows the description of many different families of functions, the descriptive power of the function describing the electron density is much larger than using a conventional family of functions. For instance, neural networks are known to be *universal approximators:* provided enough neurons and layers, they can target any function [20].

A first example of density prediction might be the one of Pande and Sinitskiy [10]. Its goal is to map a numerical tensor of 3D densities ($\{\rho^{hf}_{ijk}\}$) coming from a *cheap* electronic structure method (HF-3c) to a correction tensor ($\{\Delta\rho^{dft}_{ijk}\}$) of 3D DFT-like densities (PBE0/pcS-3):

$$\left\{\rho^{hf}_{ijk}\right\} \overset{\mathcal{F}}{\rightarrow} \left\{\Delta\rho^{dft}_{\mathrm{ijk}}\right\} \tag{7}$$

Subindexes *ijk* are kept here to remark the 3D nature of the tensors, necessary for convolution operations. Somehow, we can compare this method to an auto-encoder making a denoising process: the original sample has noise because of an uncorrelated wave function and a limited basis set, and the method will remove such noise, including then a more complete basis set and electronic correlation. Besides, those authors also built their method to allow regressing the energy difference between the uncorrelated calculation and the correlated calculation.

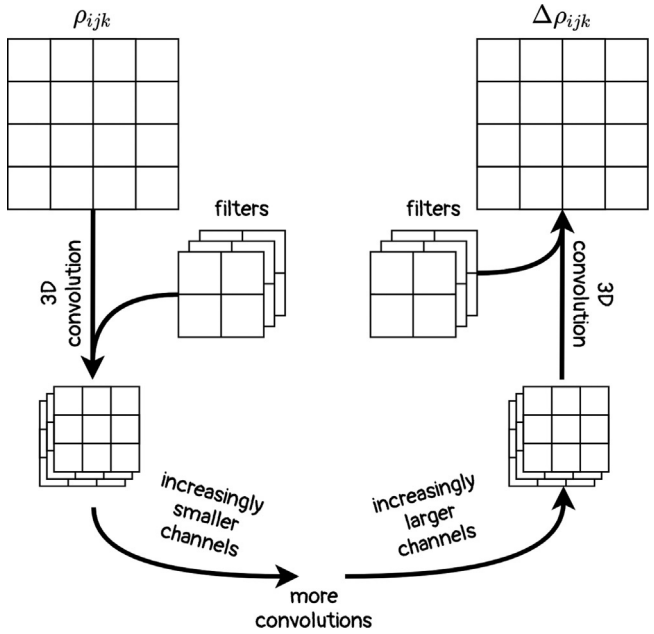


FIG. 3 Scheme of the method by Pande and Sinitskiy. 3D convolutional neural networks employ a collection of filters to turn a tensor representation of the electron density into a set of different smaller tensors. The process is continued until a given point where the representation is turned into a large set of minimal size channels. Then, another set of convolutions turns the channels into larger representations until the final representation, which has the original size and represents the corrected electron density. *Credit: Bruno Cuevas.*

The implementation of this map is based on convolutional neural networks (described previously in Chapter *Neural networks*). The potential applications of convolutions on electron density are large, but they are shadowed by some representation issues, such as discrete sampling or rotational variance (Fig. 3).

A more direct approach is that followed by DeepDFT [21], which is based on similar previous work on a different graph architecture [22]. These methods are fed with the molecular structure; they leverage their graph structure by using either a graph-convolution approach or a message-passing approach and finally employ a neural network to generate the electron density in 3D coordinates.

DeepDFT allows the representation of the 3D space as a graph by considering two kinds of nodes: atom coordinates as atomic nodes and query coordinates as probe nodes. Atomic nodes exchange information from one to the other (Fig. 4A), whereas probe nodes only receive messages from the atomic nodes. The exchange of messages is carried out along several rounds. The density at query coordinates is then read out from the probe node states.

The results are quite remarkable: they have managed to perform ML calculations in systems containing thousands of atoms, including transition metals, with errors below what different DFT functionals would have provided.

These methods can be included in DFT methodologies aimed at real-valued density grids, such as those implemented within SIESTA [23].

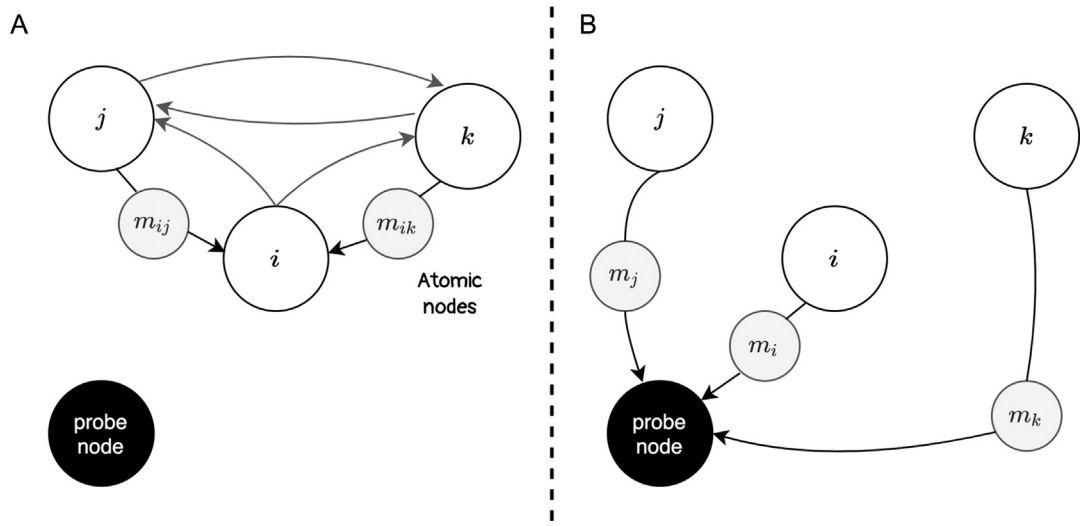


FIG. 4 (A) Bidirectional message passing between atomic nodes. Nodes create messages that include information between the sender and receiver nodes, along with the edge features between them. Only the messages sent to node i are depicted to avoid cluttering. (B) Unidirectional message passing. Probe nodes do not send any information, but they receive it from the other nodes. *Credit: Bruno Cuevas.*

Analytical methods

Analytical electron densities predict $\rho(\mathbf{r})$ in the function space. They build a generalized ansatz of the electron density based on the location of the atoms and their respective atomic numbers. The ML prediction is performed on the parameters associated to the functions: most of the time, the coefficients (c_i) associated to a linear expansion of $\chi_i(\mathbf{r})$ functions:

$$\rho(\mathbf{r}) = \sum_i c_i(M)\chi_i(\mathbf{r}) \tag{8}$$

ML research efforts are shifting from black-box prediction of the ground-state potential energy hypersurface to predicting the electronic structure of ground and excited states. Therefore, some new methods, such as SchOrbNet [18], allow the prediction of fully QM-flavored electron density from ML-predicted wave functions. We shall begin by the methods aiming directly to predict $\rho(\mathbf{r})$, and we will only mention those other methods that allow the prediction on their quests to predict some other properties (ground-state energies, wave functions, correlation energies, etc.).

Symmetry-adapted Gaussian process regression

Symmetry-adapted Gaussian process regression (SA-GPR) [15,16,24] describes $\rho(r)$ in terms of a linear expansion of a standard Gaussian basis set:

$$\rho(r) = \sum_i \sum_n \sum_l \sum_{m=-1}^{l} c^i_{nlm} g_n(r - r_i) \mathrm{Y}^m_l \left(\frac{r - r_i}{|r - r_i|} \right) \tag{9}$$

Although this representation is very convenient from a computational point of view, it is not invariant upon rotation. If we apply a rotation $\widehat{R}$ to the coordinates, the coefficients c^i_{nlm} are not valid anymore, and they need to be updated by applying the corresponding Wigner $D^l(r)$ matrix:

$$c'_{nlm} = \sum_{m'} D^l_m \left(\widehat{R} \right) c_{nlm} \tag{10}$$

An ML method aiming to make accurate representations of the coefficients c must be able to separate the effect of rotations on the values of the coefficients. SA-GPR does so by integrating the SOAP representation of overlapping pseudo-electron densities over the angular domain (rotation $\widehat{R}$):

$$k^\lambda(x, x') = \int d\widehat{R}\ \mathbf{D}^\lambda \left(\widehat{R} \right) \left| \int d\mathbf{r}\ \rho(\mathbf{r}) \rho' \left(\widehat{R}\mathbf{r} \right) \right|^2 \tag{11}$$

where $\rho(\mathbf{r})$ are atom-centered functions of Eq. 9, λ is the irreducible tensor representation, and $\mathbf{D}^\lambda$ is the Wigner D matrix corresponding to such irreducible representation. Note that when $\lambda = 0$, we recover the formulation of SOAP. The integrals can be solved analytically to map the similarity between local atomic environments and project their similarities on their corresponding coordinate frames. By using these features to feed a Gaussian Process Regression, we recover a method for prediction of the electron density in the form:

$$\rho(r) = \sum_i \sum_j^{\mathcal{M}} \sum_{nlm} w^j_{nlm}\ k^l_{mm'}(x_i, x_j)\ \ \chi_{nlm}(\mathbf{r} - \mathbf{r}_i) \tag{12}$$

where w^j_{nlm} are the weights associated to the Gaussian Process Regression of each example of the training set ($\mathcal{M}$). In the original application of SA-GPR to predict the electron density, the method learnt from a squared loss functional on the reproduction of $\rho(\mathbf{r})$. Later work replaced the direct learning of the electron density by the learning of Resolution Identity coefficients. The results could be compared with DeepDFT according to their respective articles, yet the benchmarks of both methods seem different.

A2MDnet and A3MDnet

A2MDnet and A3MDnet [25,26], are both electron density prediction approaches based on neural networks and analytic function sets. Although SA-GPR allows the definition of invariant $\rho(\mathbf{r})$ representation in spherical harmonic space, A3MDnet employs a simpler definition of electron density deformations because of the interactions between pairs of atoms (Fig. 5).

The result is a set of analytic functions that allows easy calculation of one-center integrals. The first model, A2MDnet, employs a symmetry-based high-dimensional neural network [27] to generate a set of coefficients for isotropic electron density functions and combines information belonging to different atoms to generate the coefficients of anisotropic electron density functions defined on the topology of a connectivity graph. A3MDnet is a methodological step

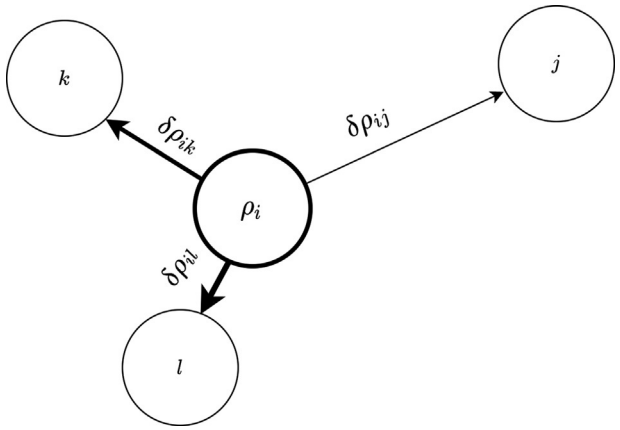


FIG. 5 Electron density representation in A2MDnet and A3MDnet. The atomic electron density (ρ) is deformed by the action of nearby atoms ($\delta\rho$). *Credit: Bruno Cuevas.*

forward which replaces the definition of a prior connectivity graph and employs a message-passing-like framework to define such a graph. Both are based on Slater functions [28], so the training of the method depends on the numerical integration of the loss functional.

Methods targeting DFT

Methods targeting DFT formalisms must deal with electronic density at some point. We can find two kinds of approaches regarding how they deal with electron density: some methods directly model $\rho(\mathbf{r})$, whereas others use it as a stream of information to estimate density functional energies. The Hohenberg–Kohn map [14] represents an example of the former one, whereas the NeuralXC ML functionals represent the second approach [17,29]. Because this book includes Chapter 23 on *Redesigning density functional theory with machine learning*, we will just consider here the aspects regarding the electron density prediction, and we will leave out aspects such as the learning of density functionals.

KH-maps

Kohn–Hohenberg theorems proved that the energy of a system is a functional of the electron density, determined by an external potential. Besides, they proved that the ground-state electron density is the minimum of such a functional. The ML Hohenberg–Kohn map joins an external potential with its associated electron density [14]:

$$\rho[\mathrm{v}](\mathbf{r}) = \sum_l^L u^l[\mathrm{v}]\chi_l(\mathbf{r}) \tag{13}$$

In this expression, $u^l[\mathrm{v}]$ is the Kernel-Ridge-Regression predictor depending on the potential v, $\chi_l(r)$ is the density basis function, and v is a vector of the external potential $v(r)$.

$$v(\mathbf{r}) = \sum_i^{atoms} Z_i \exp\left(-\frac{|\mathbf{r}-\mathbf{r}_i|}{2\gamma^2}\right) \tag{14}$$

calculated on a grid of points around the molecule. The authors employed a Fourier series (plane waves) as $\rho(\mathbf{r})$ basis, and the resulting electron density was used to map the total energy

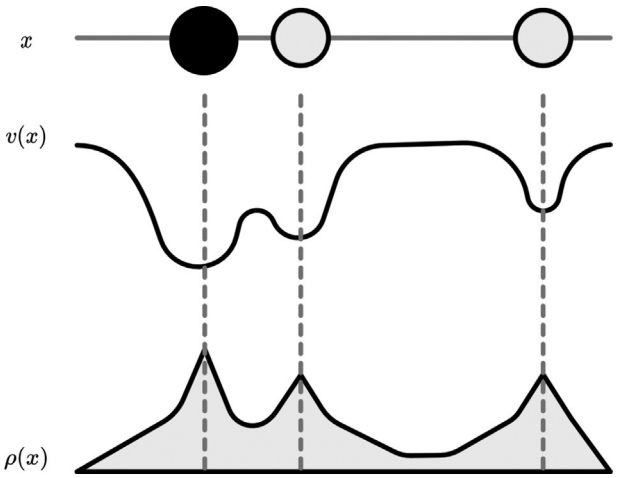


FIG. 6 Hohenberg–Kohn map diagram in 1D. Nuclei generate a given potential v(x) as a sum of Gaussians. According to the HK theorem, the potential determines the electron density, so the HK map employs the potential to parameterize a linear planar-wave basis set expansion. *Credit: Bruno Cuevas*

employing another ML predictor. The strategy employed to predict total energy from ML density is later exploited to correct DFT energies based on their densities, allowing to reach chemical accuracy [30] (Fig. 6).

The *Quantum Deep Field* (QDF) is a method related to the HK maps [9]. The later ones bypass the Kohn–Sham formalism by making a one-shot prediction of the electron density. QDF aims to predict the energy from the values of atomic orbitals in a spatial grid by building a map between atomic positions and molecular orbitals and a second map between molecular orbitals and energy:

$$\{Z, r\} \overset{\mathcal{F}_{mo}}{\rightarrow} \{\psi\} \tag{15a}$$

$$\{\psi\} \overset{\mathcal{F}_{ene}}{\rightarrow} E \tag{15b}$$

Although these two maps were successful to predict energy, the predicted molecular orbitals and their electron density were meaningless. Therefore, a third map was added to join the electron density of the molecular orbitals (15a output) with an external potential defined similar to Eq. 14:

$$\{\rho\} \overset{\mathcal{F}_{pot}}{\rightarrow} \{v\} \tag{16}$$

It is worth noticing that the third map is somehow an inverse Hohenberg–Kohn map. The outcome of the interaction between the three maps was a regularized energy predictor, so the energies could be learnt from a minimum dataset and extrapolated to a large dataset. The method predicts qualitative aspects of the electron density, which is a milestone considering that the training process *never* shows any electron density to the method.

XC functionals

Other DFT-based methods generate electron densities because of their role in DFT self-consistent calculations. NeuralXC [17] aims to generate a density functional by learning

the relation between electron density and energy. First, it decomposes an electron density obtained through some DFT calculations by projecting the density into a tailored basis set. Then, it combines the decomposition coefficients with symmetry functions in a neural network. The model is trained to reproduce the energy, so it allows the calculation of the potential of the electron density by backpropagation. This approach can be included to improve DFT calculations; this along with other approaches is reviewed in Chapter 23 [31,32].

Wave function prediction

Finally, we can include here SchNOrb [18] as an example of indirect electron density prediction. SchNOrb modifies the SchNet architecture [33] (Deep Tensor [34] Continuous convolutions) to predict the elements of the Hamiltonian operator. By replacing QM integrals of different operators between basis set functions by ML predictions, the method replaces the most expensive operation of QM calculations, so the molecular orbitals can be obtained by diagonalizing the resulting Hamiltonian matrix. The result is an ML wave function, from which electron density can be obtained directly.

Electron density methods

We shall present here a few aspects related with electron density: how to calculate it from a wave function and how to integrate it.

3D space integration

Apart from a Monte-Carlo approach, the easiest way to perform a standard 3D integral is by using a Cartesian integration grid:

$$\int f(\mathbf{r})d\mathbf{r} = \sum_i^{N_x}\sum_j^{N_y}\sum_k^{N_z} f\left(\mathbf{r}_{ijk}\right)\, x \cdot y \cdot z \tag{17a}$$

$$\mathbf{r}_{ijk} = \mathbf{r}_0 + (i\mathbf{x} + j\mathbf{y} + j\mathbf{z}) \tag{17b}$$

where $f(\mathbf{r})$ is the function we aim to integrate; $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ are the vectors of the grid; and $\mathbf{r}_0$ the origin of the cartesian grid. However, the most important factor that shapes electron density is the external potential, which depends on the distance to nuclei. Therefore, most of the electron density is arranged spherically around the nuclei, making Cartesian grid integration a bad choice in terms of convergence (number of points vs accuracy).

Spherical integration considers two integration variables: the distance (r) and the orientation (ω).

$$\int f(r)dr = \sum_i^{N_r}\sum_\omega^{N_\omega} f(r_i \cdot \mathbf{r}_\omega) \cdot w_{i\omega} \tag{18}$$

Here, the weight of each integration point depends on the value of i and ω. However, we can still split the contributions of the distance and orientation as

$$w_{i\omega} = w_i \cdot w_\omega \tag{19}$$

Therefore, we can consider this integration scheme as two independent integration schemes. We shall focus on each of them separately. The radial integration can be computed

in terms of Euler or Simpson integration. However, these approaches are inefficient in terms of convergence. A smarter choice is the use of Gauss–Chebysev quadrature:

$$\int_{-1}^{1} f(x)dx \approx \sum_{i}^{n} f(x_i)\, w_i \tag{20a}$$

$$x_i = \cos\left(\frac{2i-1}{2n}\pi\right) \tag{20b}$$

$$w_i = \frac{\pi}{n}\sqrt{1-x^2} \tag{20c}$$

From Eq. (20a), we can find that we need to adapt r to span the range of x, which is possible using

$$r = r_m \frac{1+x}{1-x} \tag{21}$$

where r_m is the mean radius parameter. It is convenient that the mean radius stays close to the atom radius.

For the orientation integral, building an accurate integration grid is much more difficult. Therefore, the standard procedure consists of reading precalculated integration grids. Among these, we might find the Lebedev grids, which allow exact integration of any polynomial on the sphere until degree n. An exhaustive list of Lebedev integration grids can be obtained at link (https://people.sc.fsu.edu/~jburkardt/datasets/sphere_lebedev_rule/sphere_lebedev_rule.html). From the files of this link, we recover a list of θ_ω, ϕ_ω, *and* w'_ω values that allow the calculation of unit orientation vectors $\mathbf{r}_\omega$.

$$r_\omega = (x,y,z) \rightarrow \begin{cases} x = \cos(\phi_\omega)\cdot \sin(\theta_\omega) \\ y = \sin(\phi_\omega)\cdot \sin(\theta_\omega) \\ z = \cos(\theta_\omega) \end{cases} \tag{22a}$$

$$w_\omega = 4\,\pi\, w'_\omega \tag{22b}$$

Joining the radial and angular integrals, we can use spherical integration grids with their respective weights. If we add the position of each of the nuclei, we can get a spherical integration grid around each nucleus. However, such an integration grid requires an additional step to down-weight the overlapping regions. Otherwise, some specific regions of space will be integrated more than once, unbalancing the integral (Fig. 7).

There are different strategies to generate such functions. A common approach depends on applying a soft-decaying function centered at nuclei and calculate the overlap as the division between the atomic function and the sum of atomic functions (switch function).

$$w_{i\omega}^{\alpha} = \frac{f(|r-r_\alpha|)}{\sum_{\alpha'} f(|r-r_{\alpha'}|)} \tag{23a}$$

$$\int f(r)dr = \sum_{\alpha}^{N_\alpha}\sum_{i}^{N_r}\sum_{\omega}^{N_\omega} f(r_i\cdot r_\omega + r_\alpha)\cdot w_{i\omega}^{\alpha}\cdot w_i \cdot w_\omega \tag{23b}$$

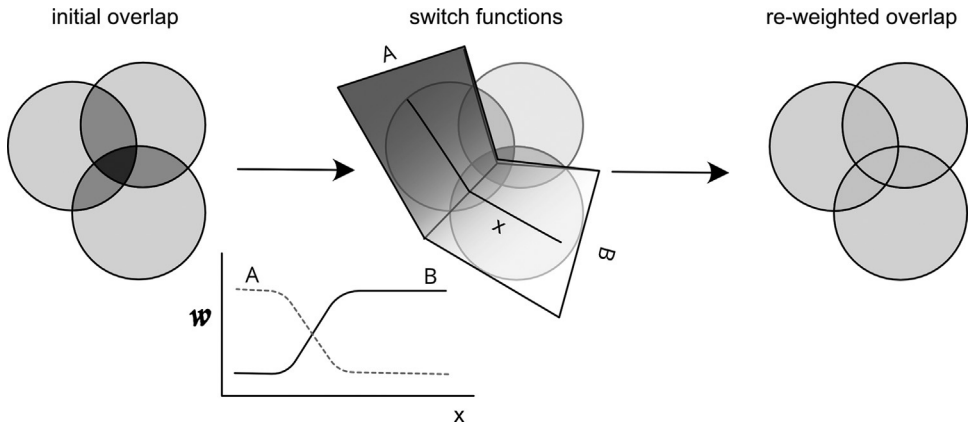


FIG. 7 Down-weight of overlapping regions. In the left, some sampling spheres have overlapping regions that are overrepresented on the integral. By applying a soft-switch function centered at each sphere $w(x)$, the weight of the points can be recalculated and reduced, so the final spheres give a uniform sampling. *Credit: Bruno Cuevas*

In the tutorial, we will use a tessellation scheme based on elliptical coordinates [35].

Electron density

Finding the electron density from QM codes is not always straight-forward. Some codes have options to output volumetric files containing the electron density. Although this option has been employed as a source to train some ML methods, it involves a numerical output which does not allow the calculation of density outside the given values unless we interpolate the values.

The calculation of the electron from a basis-set-mediated wave function is

$$\rho(\mathbf{r}) = \sum_i^{MO} n_i |\phi_i(\mathbf{r})|^2 = \sum_i^{MO} n_i \sum_j^{AO} \sum_k^{AO} c_{ij} c_{ik} \chi_j(\mathbf{r}) \chi_k(\mathbf{r}) \tag{24}$$

which can be written in a more compact notation:

$$\rho(r) = \sum_j^{AO} \sum_k^{AO} P_{jk} \chi_j(r) \chi_k(r) = \chi(r) \mathrm{P}\, \chi^T(r) \tag{25}$$

where P_{jk} represents the elements of the density matrix. Reproducing the electron density requires obtaining the parameters of the basis set functions and the values of the wave function coefficients.

The WFN format, which can be generated by many popular QM codes, allows to retrieve all the wave function information required to compute electron densities. The wave function in the WFN format is described in terms of Cartesian Gaussian-type orbitals (GTOs):

$$\chi_i(r) = i^{l_x} j^{l_y} k^{l_z} \exp\left(-\alpha\, r^2\right) \tag{26a}$$

$$i = (\mathbf{r} - \mathbf{r}_i) \cdot \mathbf{x}; \mathbf{j} = (\mathbf{r} - \mathbf{r}_\mathrm{i}) \cdot \mathbf{y}; \mathrm{k} = (\boldsymbol{r} - \mathbf{r}_i) \cdot \mathbf{z} \tag{26b}$$

$$r = |\mathbf{r} - \mathbf{r}_i| \tag{26c}$$

WFN files employ fixed size file fields to store the elements of the wave function: atomic coordinates, primitive centers, symmetries, exponents, occupancy of each molecular orbital, and the coefficients of each molecular orbital. To obtain a WFN file, we can employ different pieces of QM software. In Gaussian09, the keyword `output=wfn` writes the output to that file. In case that we have a post-Hartree Fock (HF) calculation, we need to also use the keyword `density=current` in order to have the proper electron density information written to the WFN file. In ORCA, we need to add a line `!AIM` at the beginning of the file. However, such a command will generate a WFN file that does not include virtual orbitals.

Another alternative consists of employing a modern QM code with a programming interface. In PySCF, we can calculate from atomic orbitals the electron density as well as its derivatives (see Case Study 1).

Case studies

Along the case studies, the student will learn

1. to use a QM code (PySCF) to obtain electron densities,
2. how to build a custom dataset about electron densities,
3. to train a neural network methodology (A3MDnet) to predict electron density,
4. how to use such a trained network to perform simple calculations (atomic charges, dipoles, electrostatic potential surfaces), and
5. to employ another different approach to predict electron density (DeepDFT).

The tutorials are Jupyter notebooks, and they require some previous experience with Python and common Machine-Learning/Deep-Learning libraries.

All the material can be found at https://github.com/brunocuevas/density-learning-tutorials.

Before running any of the case studies, it will be convenient to install the libraries and their dependencies. We encourage the use of conda to install all the libraries.

```
conda create –file environment.yml
```

Note: the installation of PyTorch might require the user to specify its OS and hardware.

Case study 1: Using PySCF to obtain reference electron densities

PySCF [36] offers a free, open-source, and programmatic alternative to some other popular QM codes (Gaussian, Orca, etc.). This means that we will be able to interact directly with different aspects of the calculation without tedious parsing of output log files.

In the first part of the tutorial, we will see how to make a simple QM calculation to compute the electron density. In the second part, we will see how to visualize such an electron density in a popular volumetric format, cube files. This format can be read by different programs: Avogadro, Chimera, ChimeraX, PyMol, and so on. From there, we will consider the storing of the wave function information into a standardized chemical format for electron density post-processing: the wfn format. As an advanced topic, we will cover the decomposition

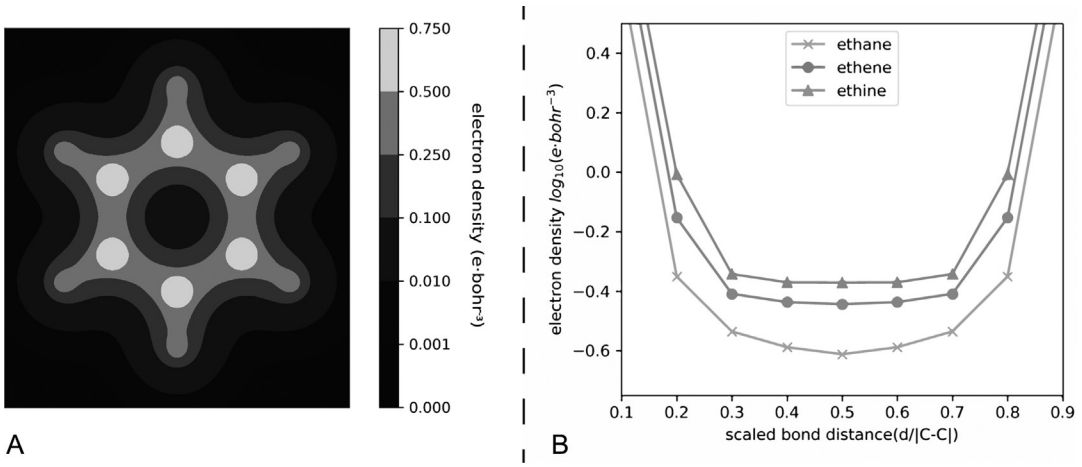


FIG. 8 (A) Example of visualization: contour plot of the electron density of a benzene molecule. (B) Example of visualization: line plot of the electron density on the bonds of ethane, ethene, and ethyne molecules. *Credit: Bruno Cuevas*

of the electron density into a linear set of functions through the resolution of identity (RI) method [37]. Finally, we will briefly show how to obtain electron density integration grids.

We propose three exercises:

Exercise 1 Use the interface for QM electron density calculations upon Cartesian coordinates to visualize the electron density in different representations:

- A contour plot for planar molecules (water, benzene)
- A line plot to visualize the electron density between two atoms
- A histogram in a random sample of points taken on a box enveloping a molecule.

Examples of such visualizations are included in Fig. 8.

Exercise 2 To test the understanding of the density calculation procedure, build a code able to parse a WFN file and compute the electron density upon sampled points.

Exercise 3 Compute properties of the electron density using the integration grid. We propose

- The calculation of the dipole of the molecules.
- Calculation of the electrostatic potential at nuclear positions
- Calculation of a simple exchange–correlation DFT functional (e.g., Thomas–Fermi kinetic energy)

Case study 2: Training an A3MDnet predictor

We will start the modeling experiment from scratch, that is, obtaining data, compiling those data, coding our predictor, and training. We will use some of the modules from A3MDnet to avoid dealing with cumbersome implementation details.

The main requisite for any ML experiment is data for training and validation. We can either employ one of the datasets available at A3MDnet Zenodo repository (), or we can create our own. Creating a training dataset consumes a lot of computational power. Therefore, we will focus on creating a specific validation benchmark to check how our predictor performs in some important biomolecules: sugars. Obviously, the experiment could be run with any other kind of molecules.

We will employ a simple search on Zinc15 [38] to find a collection of molecules containing a given sugar substructure. From those molecules, we downloaded the SMILES and we have embedded molecular coordinates. Then, we have optimized those first using the Merk Molecular Field integrated in RDKit and then employing the ANI ML potentials [27]. The resulting conformations have been employed to calculate the electron density using ORCA in a DFT calculation. We have obtained a collection of WFX files, which we have compiled to train our predictor.

Advanced exercise Build a PySCF pipeline to calculate the same wfn files using the concepts from Study Case 1.

Once we have the data, it is time to code our predictor. A3MDnet employs atomic embeddings defined at each atomic position. Such embeddings are mapped from their atomic numbers. After that, different layers of message passing might be stacked, or a given layer might be reused. Because of the usual trade-off between capacity and overfitting risk, we will employ only a given message-passing layer a given number of times. Finally, the atomic embeddings must be translated into either isotropic coefficients or anisotropic vectors. The first ones are the coefficients of the isotropic expansion

$$\rho_{iso}(\mathbf{r}) = \sum_{\alpha}^{N_\alpha} \sum_{k}^{N_f} c_k^\alpha \chi_k(\mathbf{r} - \mathbf{r}_\alpha) \tag{27}$$

where $\chi_k(\mathbf{r}-\mathbf{r}_\alpha)$ is an exponential function $\chi_k(\mathbf{r}-\mathbf{r}_\alpha) = A_k \exp(-B_k|\mathbf{r}-\mathbf{r}_\alpha|)$. The second ones define the *strength* of the anisotropic deformation (vector module, $\mathbf{c}_k^\alpha$) and the direction of such deformation:

$$\rho_{ani}(\mathbf{r}) = \sum_{\alpha}^{N_\alpha} \sum_{k}^{N_f} c_k^\alpha \chi_k\left(\mathbf{r} - \mathbf{r}_\alpha, \mathbf{c}_k^\alpha\right) \tag{28a}$$

$$\chi_k\left(r - r_\alpha, c_k^\alpha\right) = A_k|\mathbf{r} - \mathbf{r}_\alpha|^{n_k} \exp\left(-B_k|\mathbf{r} - \mathbf{r}_\alpha|\right) P_l^0\left(\mathbf{c}_k^\alpha \cdot \frac{\mathbf{r} - \mathbf{r}_\alpha}{|\mathbf{r} - \mathbf{r}_\alpha|}\right) \tag{28b}$$

where P_l^m is a Legendre polynomial. This approach makes cumbersome the analytic calculation of integrals between functions. However, it eases representation, given that it is invariant to rotation and translation. We define the electron density then as

$$\rho = \rho_{pro} + \rho_{iso} + \rho_{ani} \tag{29}$$

where ρ_{pro} is the reference pro-molecular electron density, which we obtain from a simple model consisting of a sum of exponentials parameterized on reference calculations [28]. The anisotropic functions are based on spherical harmonics, so their space integrals are zero. Therefore, it is only necessary to normalize the charge of the isotropic part, so its contribution is equal to the molecule charge:

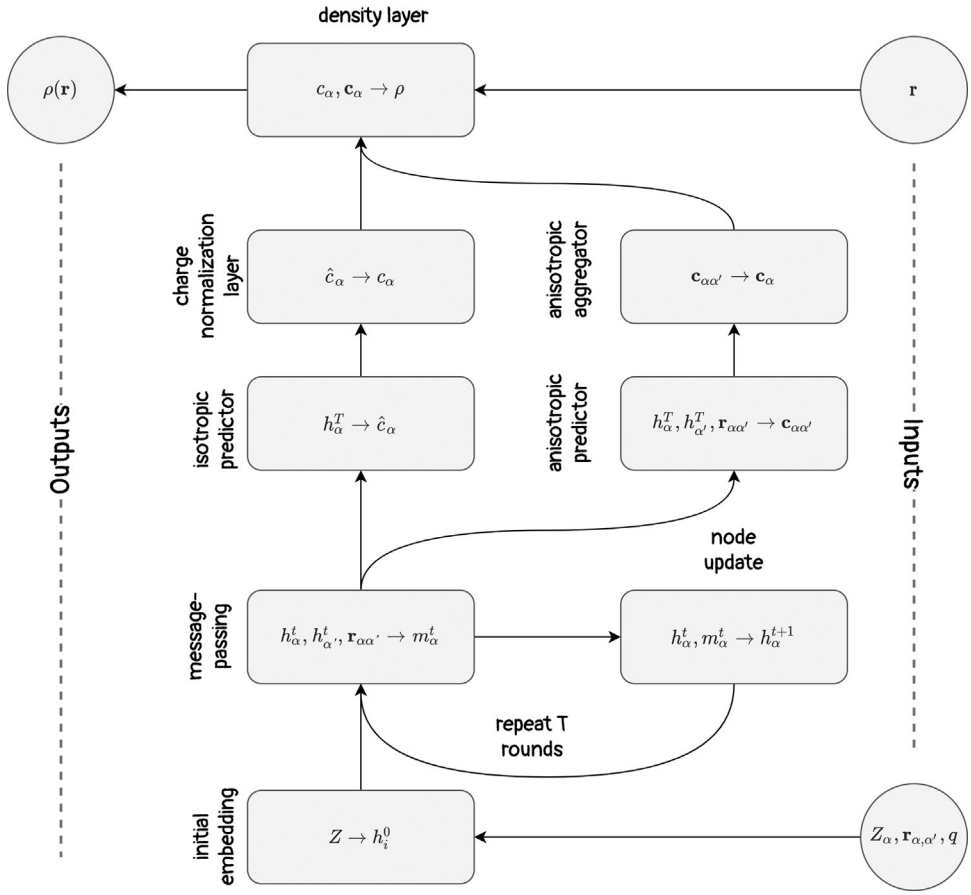


FIG. 9 Scheme of the A3MDnet architecture. *Credit: Bruno Cuevas*

$$\int \rho_{iso} dr = \sum_{\alpha}^{N_a} \sum_{k}^{N_f} c_k^\alpha \, X_k = -q \tag{30}$$

The normalization is fulfilled by projecting the coefficients into a normalization hypersurface, where the charge is normalized. The anisotropic prediction generates a set of deformations for each pair of atoms and anisotropic functions. Although a model containing a large degree of anisotropic deformations might be a powerful one, it would become expensive. Therefore, anisotropic deformations are aggregated to reduce their number to a few relevant deformations. The most successful mechanism tested depended on employing only the deformations generated by the nearest atoms (Fig. 9).

The predictor can be trained by just using a squared loss functional.

$$\mathcal{L} = \int \Big(\rho(\mathbf{r}) - \rho_{ref}(\mathbf{r})\Big)^2 d\mathbf{r} \tag{31}$$

This functional is solved numerically employing the integration grids explained in Section "3D space integration". This means that for each iteration, we must compute a set of integration grids, their associated weights, and the reference electron density.

Once trained, the models can be employed for different tasks. We propose some exercises based on what we saw in the previous tutorial:

Exercise 1: Predicting Hirshfeld charges: They are predicted by integrating the electron density by weighting the electron density by an atomic electron density divided by the total pro-molecular electron density (sum of atomic densities).

$$q_i = \int \rho(\mathbf{r}) \frac{\rho_i^{pro}(\mathbf{r})}{\sum_j \rho_j^{pro}(\mathbf{r})} d\mathbf{r} \tag{32}$$

Tip: Use the pro-molecular density of A3MDnet models to compute the weighting factor.

Exercise 2: Predicting molecule dipoles: They can be predicted by performing a numerical integral on the electron density, as in the previous case.

Case study 3: Predicting valence electron densities with DeepDFT

DeepDFT is a methodology based on message-passing neural networks for prediction of the electron density in 3D real space. This is performed by considering real coordinate probe nodes that receive messages from graph-adjacent atomic nodes. We will give a brief example based on one of the scripts provided at the DeepDFT repository (https://github.com/peterbjorgensen/DeepDFT), and we will provide an example about how to calculate electron density at custom coordinates.

Finally, we will combine the DeepDFT method and an AiM analysis to compute the charge transfer in an ionic hydrogen bond.

Conclusions and outlook

Electron density is a fascinating object of quantum chemistry. Although wave function lives in a mathematical space that our minds cannot grasp, the electron density lives in the familiar 3D space that we know.

Predicting the electron density allows us to bring physical motivation into ML: instead of training different predictors for each specific property (charges, dipoles, energies), we can directly predict a physical variable and then use that same magnitude to derive all the other targets through the application of rigorous physical approaches. In this way, properties are consistent, and the same ML output can be employed to derive properties unplanned at first. Doing so with the wave function is a milestone recently achieved, but electron density keeps being an attractive candidate because of its mathematical handiness.

Future steps should show clear applications of these methodologies in relevant problems from ML-enhanced DFT to structure elucidation. There is still room to test these algorithms to explain chemical insights combined with information theory or Atoms in Molecules theory

(as some work has been already performed with other density analysis tools [15]). In any case, the future developments in ML and the implementation of more capabilities in the standard ML frameworks will open new technical possibilities for predicting electron densities.

Acknowledgments

Thanks Dr. Luis F. Pacios for his review and comments.

References

[1] P. Hohenberg, W. Kohn, Inhomogeneous electron gas, Phys. Rev. 136 (3B) (1964) B864, https://doi.org/10.1103/PhysRev.136.B864.
[2] W. Kohn, L.J. Sham, Self-consistent equations including exchange and correlation effects, Phys. Rev. 140 (4A) (1965) A1113–A1138, doi:10.1103/PhysRev.140.A1133.
[3] R. Car, Density functional theory: fixing Jacob's ladder, Nat. Chem. 8 (9) (2016) 820–821, https://doi.org/10.1038/nchem.2605.
[4] A. Pribram-Jones, D.A. Gross, K.D.F.T. Burke, A Theory Full of Holes?, 2014, https://doi.org/10.1146/annurev-physchem-040214-121,420.
[5] P.L.A. Popelier, Atoms in Molecules: An Introduction, Prentice Hall, UK, 2000.
[6] M.A. Marques, M.D. Purdy, M. Yeager, CryoEM maps are full of potential, Curr. Opin. Struct. Biol. 58 (August) (2019) 214–223, https://doi.org/10.1016/j.sbi.2019.04.006.
[7] C.M. Phys, R. Froese, I. Herbst, M. Hoffmann-ostenhof, L2 - exponential lower bounds to solutions of the schrödinger equation, Commun. Math. Phys. 286 (4240) (1982) 265–286.
[8] T. Kato, On the eigenfunctions of many-particle systems in quantum mechanics, Commun. Pure Appl. Math. 10 (2) (1957) 151–177, https://doi.org/10.1002/cpa.3160100201.
[9] M. Tsubaki, T. Mizoguchi, Quantum deep field: data-driven wave function, electron density generation, and atomization energy prediction and extrapolation with machine learning, Phys. Rev. Lett. 125 (20) (2020) 206,401, https://doi.org/10.1103/PhysRevLett.125.206401.
[10] A.V. Sinitskiy, V.S. Pande, Deep Neural Network Computes Electron Densities and Energies of a Large Set of Organic Molecules Faster than Density Functional Theory (DFT), *arXiv*, **2018**.
[11] R.F.W. Bader, Atoms in Molecules : A Quantum Theory, Clarendon Press, Oxford, UK, 1990.
[12] T. Verstraelen, S. Vandenbrande, F. Heidar-Zadeh, L. Vanduyfhuys, V. van Speybroeck, M. Waroquier, P.W. Ayers, Minimal basis iterative stockholder: atoms in molecules for force-field development, J. Chem. Theory Comput. 12 (8) (2016) 3894–3912, https://doi.org/10.1021/acs.jctc.6b00456.
[13] C. Rong, B. Wang, D. Zhao, S. Liu, Information-theoretic approach in density functional theory and its recent applications to chemical problems, Wiley Interdiscip. Rev.: Comput. Mol. Sci. 10 (4) (2020) 1–22, https://doi.org/10.1002/wcms.1461.
[14] F. Brockherde, L. Vogt, L. Li, M.E. Tuckerman, K. Burke, K.-R. Müller, Bypassing the Kohn-Sham equations with machine learning, Nat. Commun. 8 (1) (2017) 872, https://doi.org/10.1038/s41467-017-00839-3.
[15] A. Fabrizio, A. Grisafi, B. Meyer, M. Ceriotti, C. Corminboeuf, Electron density learning of non-covalent systems, Chem. Sci. 10 (41) (2019) 9424–9432, https://doi.org/10.1039/c9sc02696g.
[16] A. Grisafi, A. Fabrizio, B. Meyer, D.M. Wilkins, C. Corminboeuf, M. Ceriotti, Transferable machine-learning model of the electron density, ACS Cent. Sci. 5 (1) (2019) 57–64, https://doi.org/10.1021/acscentsci.8b00551.
[17] S. Dick, M. Fernandez-Serra, Machine learning accurate exchange and correlation functionals of the electronic density, Nat. Commun. 11 (1) (2020), https://doi.org/10.1038/s41467-020-17,265-7.
[18] K.T. Schütt, M. Gastegger, A. Tkatchenko, K.R. Müller, R.J. Maurer, Unifying machine learning and quantum chemistry with a deep neural network for molecular wavefunctions, Nat. Commun. 10 (1) (2019) 1–10, https://doi.org/10.1038/s41467-019-12,875-2.
[19] M. Gastegger, A. McSloy, M. Luya, K.T. Schütt, R.J. Maurer, A deep neural network for molecular wave functions in quasi-atomic minimal basis representation, J. Chem. Phys. 153 (4) (2020), https://doi.org/10.1063/5.0012911.

[20] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366, https://doi.org/10.1016/0893-6080(89)90020-8.
[21] P.B. Jørgensen, A. Bhowmik, DeepDFT: Neural Message Passing Network for Accurate Charge Density Prediction, *arXiv*, 2020.
[22] S. Gong, T. Xie, T. Zhu, S. Wang, E.R. Fadel, Y. Li, J.C. Grossman, Predicting Charge Density Distribution of Materials Using a Local-Environment-Based Graph Convolutional Network, *arXiv*, **2019**, pp. 1–26.
[23] J.M. Soler, E. Artacho, J.D. Gale, A. García, J. Junquera, P. Ordejón, D. Sánchez-Portal, *The SIESTA method for* Ab initio *order-N materials*, Simulation 14 (2002).
[24] A. Grisafi, D.M. Wilkins, G. Csányi, M. Ceriotti, Symmetry-Adapted Machine-Learning for Tensorial Properties of Atomistic Systems., 2017, https://doi.org/10.1103/PhysRevLett.120.036002.
[25] B. Cuevas-Zuviría, L.F. Pacios, Analytical model of electron density and its machine learning inference, J. Chem. Inf. Model. 60 (8) (2020) 3831–3842, https://doi.org/10.1021/acs.jcim.0c00197.
[26] B. Cuevas-Zuviría, L.F. Pacios, Machine learning of analytical electron density in large molecules through message-passing, J. Chem. Inf. Model. (2021), https://doi.org/10.1021/acs.jcim.1c00227. acs.jcim.1c00227.
[27] J.S. Smith, O. Isayev, A.E. Roitberg, ANI-1: an extensible neural network potential with dft accuracy at force field computational cost, Chem. Sci. 8 (4) (2017) 3192–3203, https://doi.org/10.1039/C6SC05720A.
[28] L.F. Pacios, Analytical density-dependent representation of Hartree-Fock atomic potentials, J. Comput. Chem. 14 (4) (1993) 410–421, https://doi.org/10.1002/jcc.540140405.
[29] S. Dick, M. Fernandez-Serra, Learning from the Density to Correct Total Energy and Forces in First Principle Simulations, 2018, https://doi.org/10.1063/1.5114618.
[30] M. Bogojeski, L. Vogt-Maranto, M.E. Tuckerman, K.R. Müller, K. Burke, Quantum chemical accuracy from density functional approximations via machine learning, Nat. Commun. 11 (1) (2020), https://doi.org/10.1038/s41467-020-19,093-1.
[31] R. Nagai, R. Akashi, O. Sugino, Completing density functional theory by machine learning hidden messages from molecules, npj Comput. Mater. 6 (1) (2020), https://doi.org/10.1038/s41524-020-0310-0.
[32] K. Ryczko, D.A. Strubbe, I. Tamblyn, Deep learning and density-functional theory, Phys. Rev. A 100 (2) (2019), https://doi.org/10.1103/PhysRevA.100.022512.
[33] K.T. Schütt, H.E. Sauceda, P.J. Kindermans, A. Tkatchenko, K.R. Müller, SchNet—a deep learning architecture for molecules and materials, J. Chem. Phys. 148 (24) (2018), https://doi.org/10.1063/1.5019779.
[34] K.T. Schütt, F. Arbabzadah, S. Chmiela, K.R. Müller, A. Tkatchenko, Quantum-Chemical Insights from Deep Tensor Neural Networks., 2016, https://doi.org/10.1038/ncomms13890.
[35] A.D. Becke, A multicenter numerical integration scheme for polyatomic molecules, J. Chem. Phys. 88 (4) (1988) 2547–2553, https://doi.org/10.1063/1.454033.
[36] Q. Sun, T.C. Berkelbach, N.S. Blunt, G.H. Booth, S. Guo, Z. Li, J. Liu, J.D. McClain, E.R. Sayfutyarova, S. Sharma, S. Wouters, G.K.-L. Chan, PySCF: the python-based simulations of chemistry framework, Wiley Interdiscip. Rev.: Comput. Mol. Sci. 8 (1) (2018), https://doi.org/10.1002/WCMS.1340, e1340.
[37] B.I. Dunlap, Robust and variational fitting, Phys. Chem. Chem. Phys. 2 (10) (2000) 2113–2116, https://doi.org/10.1039/b000027m.
[38] T. Sterling, J.J. Irwin, ZINC 15–ligand discovery for everyone, J. Chem. Inf. Model. 55 (11) (2015) 2324–2337, https://doi.org/10.1021/acs.jcim.5b00559.